



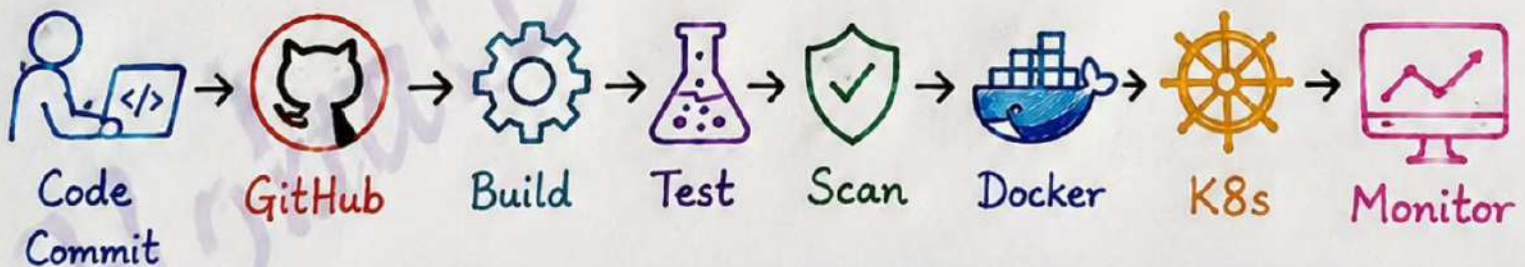
Slide 1.

≡ COMPLETE ≡

# CI/CD

## PIPELINE EXPLAINED

From Code Commit  
to Production Deployment



Automate • Test • Secure • Deploy • Monitor

☆ Deliver Quality Software  
Faster & Smarter!

By Nagarani Sunke ❤️

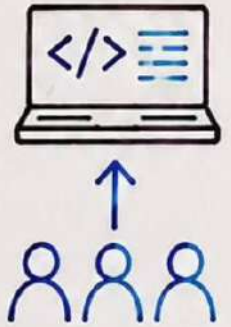


# WHAT IS CI / CD?



## CI (CONTINUOUS INTEGRATION)

- Developers frequently merge code.
- Every commit is automatically built and tested.



## CD (CONTINUOUS DELIVERY / DEPLOYMENT)

- Automatically deploys validated code.
- Delivers software faster with fewer errors.



GOAL: Build once, deploy anywhere.



By Nagarani Sunke.



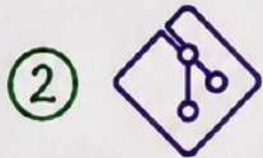


## ≡ STEP 1-4 ≡

# DEVELOPMENT PHASE



① Developer



Git Commit



Push to GitHub



Webhook  
Trigger

① Developer writes code

- Developer creates or updates features / fixes bugs in the application.

② Git Commit

- Changes are committed to the local Git repository with a meaningful message.

③ Push to GitHub

- Code is pushed to remote repository (GitHub).

④ Webhook Trigger

- GitHub sends a webhook notification to CI/CD tool (Jenkins / GitHub Actions). This automatically triggers the pipeline.



Every code push automatically starts the CI/CD pipeline.

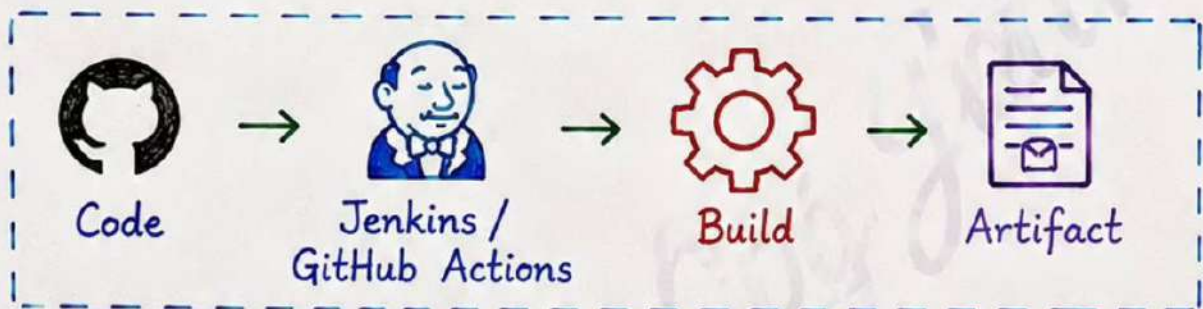


By Nagarani Sunke.



## STEP 5

# BUILD STAGE



## BUILD (MAVEN / GRADLE)

The CI tool uses build tools to compile the code and produce an executable artifact.

### What happens?



- ✓ Download dependencies
- ✓ Compile source code
- ✓ Run validations
- ✓ Package the application
- ✓ Generate Artifact (JAR/WAR)

### Tools Used

- Maven 
- Gradle 

### Output

Application Artifact



This artifact will be used to create a Docker image in the next step.

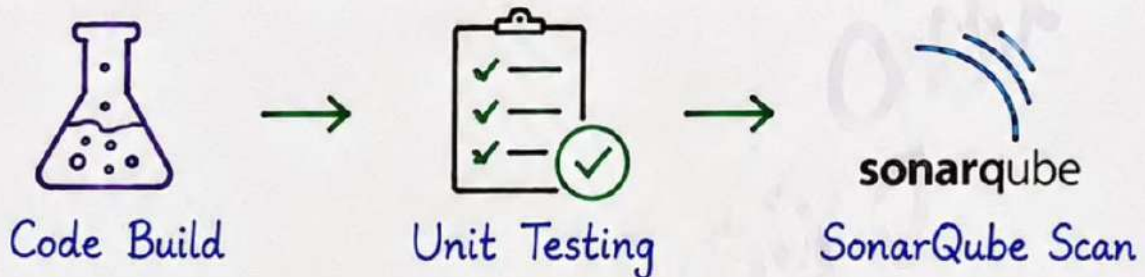


♥ By Nagarani Sunke ♥



≡ STEP 6-7 ≡

# TESTING & QUALITY



## ① UNIT TESTING

- Automatically tests individual components of the application.
- Ensures the code works as expected.
- If any test fails, the pipeline stops here.



## ② SONARQUBE SCAN

Performs static code analysis to check:

- ☒ Bugs
- ☒ Code Smells
- ☒ Vulnerabilities
- ☒ Code Coverage
- ☒ Duplicated Code

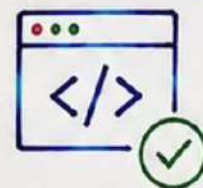


Quality Gate  
PASS / FAIL

### Why is this important?



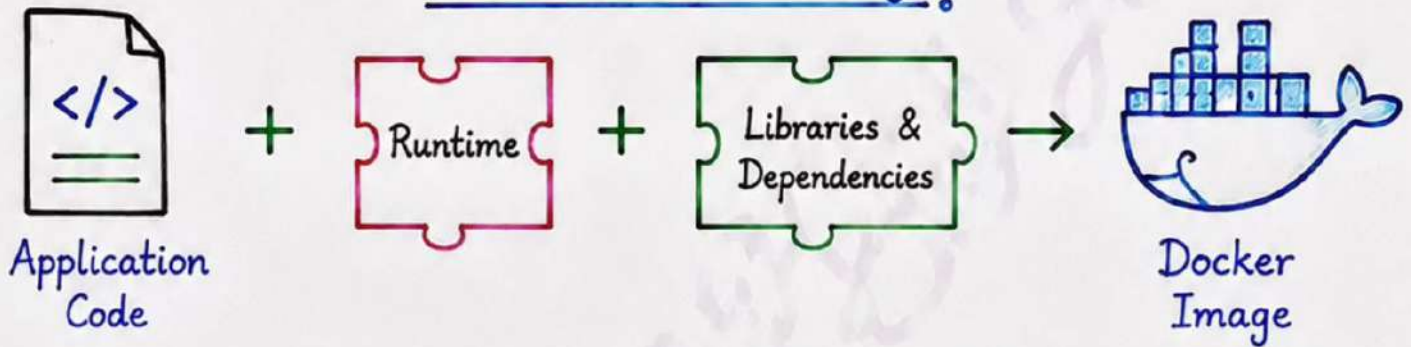
Catching issues early makes the code clean, reliable and production ready.



## STEP 8

# CONTAINERIZATION

### Build Docker Image



### What is a Docker Image?

A Docker image is a lightweight, standalone, executable package that includes everything needed to run an application: code, runtime, libraries, environment variables and configuration.



### Benefits

- ✓ Same image runs anywhere
- ✓ Consistent across all environments
- ✓ Easy to version and rollback
- ✓ Fast, lightweight and portable



Output: Docker Image (e.g., `myapp:1.0.0`)



By Nagarani Sunke





## STEP 9 SECURITY SCAN

### Trivy Scan (Docker Image)



Docker Image



Trivy Scan



Vulnerability Report

#### What does Trivy Scan?

- ✓ Detects OS package vulnerabilities
- ✓ Scans application dependencies
- ✓ Finds secrets in images
- ✓ Checks misconfigurations
- ✓ Provides severity details (Critical, High, Medium, Low)

#### Severity Levels

- Critical Fix immediately
- High Fix as soon as possible
- Medium Fix in next iteration
- Low Minor issues

#### Why it matters?



Catching vulnerabilities early prevents security risks in production.



If Critical / High vulnerabilities are found, the pipeline fails and stops.



By Nagarani Sunke.

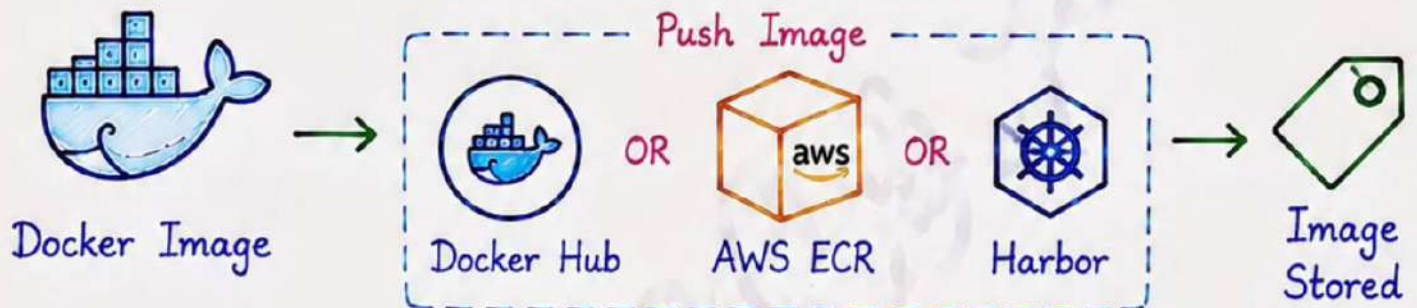




## STEP 10

# CONTAINER REGISTRY

### Store Docker Image



### Image Tagging



- v1.0.0 → Stable Release
- v1.0.1 → Bug Fix
- latest → Latest Build
- dev → Development Build

### Why Use a Registry?

- ☒ Centralized place to store images
- ☒ Share images across teams
- ☒ Version control with tags
- ☒ Pull images in deployments
- ☒ Secure and reliable storage



Once the image is stored in the registry,  
it can be pulled by Kubernetes  
for deployment.



### Output:

Versioned and secure Docker Image  
ready for deployment.



By Nagarani Sunke.





## STEP 11

# DEPLOY TO KUBERNETES

### Deploy to Kubernetes Cluster.



### What Happens?



- Kubernetes pulls the image from the registry.
- Creates/updates Pods as per the manifest.
- Performs rolling update (zero downtime).
- Ensures desired state is maintained.

### Key Points

- ✓ Rolling update for zero downtime
- ✓ Auto healing & self recovery
- ✓ Scalable & reliable
- ✓ Health checks ensure only healthy pods receive traffic

### Why it matters?



Kubernetes ensures your application is always available, scalable and self-healing.



Output: Application is deployed and running in Kubernetes Pods (Healthy & Ready).



### Health Check

- ✓ Pods Running
- ✓ Containers Healthy
- ✓ Service Available



By Nagarani Sunke

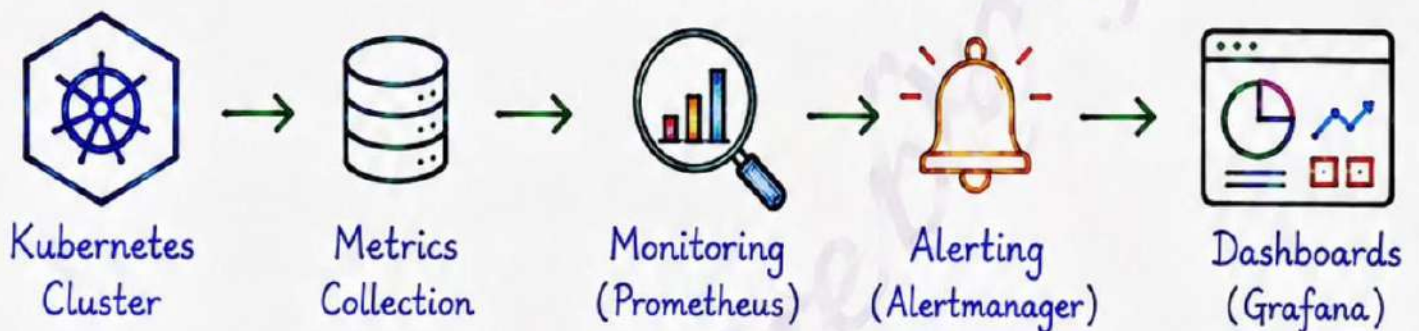




## STEP 12

# MONITORING & ALERTING

Monitor your applications and infrastructure to ensure reliability and performance.



### What is Monitoring?



- Continuously collects metrics from your applications and infrastructure.
- Helps in tracking performance, availability and health.
- Provides visibility into issues before users are affected.

### Key Components

- **Prometheus** : Collects & stores metrics
- **Alertmanager** : Handles alerts
- **Grafana** : Visualizes metrics with dashboards
- **Node Exporter** : Collects host metrics
- **Kube State Metrics** : Collects Kubernetes object metrics

### Why it matters?



Monitoring ensures early detection, faster troubleshooting and better system reliability.



- Detect Issues Early
- Reduce Downtime
- Improve Performance
- Better User Experience



**Output:** Real-time visibility into your system





## STEP 13

# DEVOPS CULTURE & BEST PRACTICES

People, Process and Tools working together.



### Collaboration

Work together as a team.



### Communication

Open, transparent and continuous.



### Continuous Improvement

Always learn, always improve.



### Security Mindset

Build security in everything.



### Deliver Value Faster

Focus on customer value delivery.

### Best Practices

- ✓ Automate Everything
- ✓ Infrastructure as Code (IaC)
- ✓ Continuous Integration & Delivery (CI/CD)
- ✓ Version Control for Everything
- ✓ Monitoring & Logging
- ✓ Security at Every Step (Shift Left)
- ✓ Small Changes, Frequent Releases

### DevOps Culture Benefits

- Higher Quality Software
- Faster Time to Market
- Reliable & Scalable Systems
- Happier Teams
- Better Customer Satisfaction



### Remember!

DevOps is not a destination, it's a continuous journey of learning, adapting and delivering value.



Output: A strong DevOps culture leads to





## STEP 14

# CAREER GROWTH & NEXT STEPS

Keep Learning. Keep Building. Keep Growing.



Learn

Keep upskilling and explore new technologies.



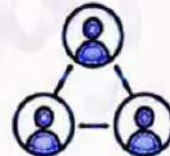
Build

Work on real world projects and contribute.



Certify

Get certified in key tools and cloud platforms.



Network

Connect with communities and mentors.

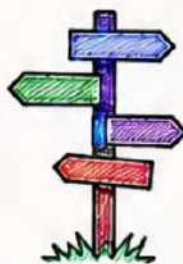


Grow

Aim for roles that challenge and inspire you.

### Career Paths in DevOps

- DevOps Engineer
- Site Reliability Engineer (SRE)
- Cloud Engineer
- Platform Engineer
- DevSecOps Engineer
- Infrastructure Architect
- Lead / Manager



### Next Steps

- ☒ Identify your strengths
- ☒ Set short term & long term goals
- ☒ Build your personal brand
- ☒ Share knowledge (write, speak, teach)
- ☒ Stay updated with industry trends
- ☒ Be consistent and patient



### Key Takeaways



- DevOps is a journey, not a destination.
- Every skill you learn adds value.
- Focus on learning, building, sharing and growing.
- Your future in DevOps is bright!

